

IMDB Sentiment Analysis using Deep Neural Network

1. Install Required Libraries

```
!pip install numpy pandas matplotlib seaborn scikit-learn tensorflow keras
```

2. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from keras.datasets import imdb
from keras import models
from keras import layers
import tensorflow as tf
```

3. Load IMDB Dataset

```
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)
```

4. Combine Dataset

```
data = np.concatenate((X_train, X_test), axis=0)
label = np.concatenate((y_train, y_test), axis=0)
```

5. Check Dataset Shapes

```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

6. Print Sample Review

```
print(X_train[0])
print(y_train[0])
```

7. Load Vocabulary

```
vocab = imdb.get_word_index()
print(vocab)
```

8. Vectorization Function

```
def vectorize(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))

    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1

    return results
```

9. Split Dataset

```
test_x = data[:10000]
test_y = label[:10000]

train_x = data[10000:]
train_y = label[10000:]
```

10. Dataset Information

```
print("Categories:", np.unique(label))
print("Number of unique words:", len(np.unique(np.hstack(data))))
```

11. Average Review Length

```
length = [len(i) for i in data]

print("Average Review length:", np.mean(length))
print("Standard Deviation:", round(np.std(length)))
```

12. Decode Review

```
index = imdb.get_word_index()

reverse_index = dict([(value, key) for (key, value) in index.items()])

decoded = " ".join([reverse_index.get(i - 3, "#") for i in data[0]])

print(decoded)
```

13. Vectorize Data

```
data = vectorize(data)
label = np.array(label).astype("float32")
```

14. Plot Label Count

```
labelDF = pd.DataFrame({'label': label})

sns.countplot(x='label', data=labelDF)
plt.show()
```

15. Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    data,
    label,
    test_size=0.20,
    random_state=1
)
```

16. Check Shapes

```
print(X_train.shape)
print(X_test.shape)
```

17. Build Neural Network

```
model = models.Sequential()
```

```

model.add(layers.Dense(50, activation="relu", input_shape=(10000,)))
model.add(layers.Dropout(0.3))

model.add(layers.Dense(50, activation="relu"))
model.add(layers.Dropout(0.2))

model.add(layers.Dense(50, activation="relu"))

model.add(layers.Dense(1, activation="sigmoid"))

model.summary()

```

18. Early Stopping

```

callback = tf.keras.callbacks.EarlyStopping(
    monitor='loss',
    patience=3
)

```

19. Compile Model

```

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

```

20. Train Model

```

results = model.fit(
    X_train,
    y_train,
    epochs=2,
    batch_size=500,
    validation_data=(X_test, y_test),
    callbacks=[callback]
)

```

21. Validation Accuracy

```

print(np.mean(results.history["val_accuracy"]))

```

22. Evaluate Model

```

score = model.evaluate(X_test, y_test, batch_size=500)

print('Test loss:', score[0])
print('Test accuracy:', score[1])

```

23. Plot Accuracy Graph

```

plt.plot(results.history['accuracy'])
plt.plot(results.history['val_accuracy'])

plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')

plt.legend(['Train', 'Test'], loc='upper left')

plt.show()

```

24. Plot Loss Graph

```
plt.plot(results.history['loss'])
plt.plot(results.history['val_loss'])

plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')

plt.legend(['Train', 'Test'], loc='upper left')

plt.show()
```